



BLOCK VENTURE CHAIN CAPITAL

Agent Wallet Informe de Seguridad y Pruebas

Análisis estático, remediación de vulnerabilidad y batería de pruebas de permisos del agente on-chain

OBJETO	BVCC Agent Wallet (smart account ERC-4337 / ERC-7579)		
RED	Arbitrum Sepolia (421614) · Factories V2 también en Arbitrum One (42161) y BNB Chain (56)		
WALLET PARCHEADA	0x1ED261BA084c4E6117daf289f3ca233BBA3cF0C3		
TRABAJO	Pruebas de seguridad internas y verificación de la remediación		
FECHAS	2026-06-06 → 2026-06-11		
ESTADO	2 HIGH detectados Corregidos y redesplegados 11/11 pruebas OK Recovery verificado V2 multichain		

1 Resumen ejecutivo

Este informe documenta las pruebas de seguridad de la BVCC Agent Wallet — una smart account ERC-4337 que permite al propietario delegar una capacidad de gasto acotada a un agente autónomo (EOA). El trabajo abarcó análisis estático, un hallazgo de severidad alta con su remediación, el redesplicue de los contratos parcheados y la re-verificación completa del sistema de permisos del agente sobre el bytecode parcheado. Una segunda ronda (V2) corrigió un problema de gas en mainnet en la lógica de snapshot de comisiones y concluyó con un despliegue multichain determinista (Arbitrum One, BNB Chain, Arbitrum Sepolia — ver §9).

2

HALLAZGOS ALTOS
(CORREGIDOS)

131/131

TESTS UNITARIOS
(FORGE)

11/11

PRUEBAS ON-CHAIN
DEL AGENTE

0

INCIDENCIAS
ABIERTAS

Resultados clave

- **Vulnerabilidad de severidad alta detectada y corregida.** Un bypass de approve permitía al agente mover tokens más allá de sus presupuestos diario/total vía `transferFrom`. Reproducible on-chain, parcheado en el código y re-verificado tras el redesplicue.
- **Problema de gas en mainnet corregido (V2).** Los precompiles de Arbitrum detectados por el escaneo de calldata del snapshot de comisiones quemaban todo el gas reenviado, inflando las estimaciones hasta ~32M y rompiendo los swaps. Cada sonda `balanceOf` está ahora capada en gas (`PROBE_GAS_CAP = 100k`); las estimaciones vuelven a ~400k para un swap real (ver §9).
- **Análisis estático limpio.** Slither 0.11.3 no produjo hallazgos accionables; se añadió un control defensivo de dirección cero a los constructores de ambas factories.
- **Contratos parcheados redesplicados** en Arbitrum Sepolia, con la clave del deployer sin privilegios (el control queda en una dirección de administración independiente).
- **La batería completa de permisos del agente pasa** sobre la wallet parcheada: por transacción, diario, presupuesto total, presupuesto por período (con auto-rollover), expiración, pausa y lista blanca de destinatarios — aplicado tanto para ETH nativo como para tokens ERC-20.

Valoración general. El modelo de permisos del agente es sólido y, tras la remediación, aplica todas las restricciones configuradas. No quedan incidencias abiertas a la fecha de redacción.

2 Alcance y metodología

En alcance

- `BVCCAgentWallet.sol` — lógica de permisos del agente (`executeAsAgent`, caps por token y ETH, pausa, expiración).
- `BVCCWallet.sol` — smart account base (ejecución, comisiones, recovery por guardianes).
- `BVCCAgentWalletFactory.sol` / `BVCCWalletFactory.sol` — factories CREATE2.

Metodología

- **Análisis estático** — Slither 0.11.3 sobre ambos repositorios.
- **Pruebas unitarias** — Foundry (`forge test`), con tests de regresión para el hallazgo.

- **Pruebas on-chain** — el EOA del agente ejecuta `executeAsAgent` directamente; los `reverts` se confirman por simulación (`eth_call`) contra el estado real y los casos válidos como transacciones reales.

Modelo de ejecución del agente

El agente llama a `executeAsAgent(bytes32 mode, bytes executionData)` como una transacción EOA ordinaria (no una `UserOp`). `mode` es el selector de batch ERC-7579 (`0x0100...00`); `executionData` es un `Execution[]` codificado en ABI de (`target`, `value`, `callData`). El orden de validación dentro de la llamada es **per-tx** → **destinatario/protocolo** → **período** → **diario** → **total**, con el estado actualizado **antes** de la ejecución externa (`checks-effects-interactions`) y un `guard nonReentrant`.

3 Análisis estático — Slither

Slither 0.11.3 se ejecutó sobre ambos repositorios antes del redesplicue. Reportó 39 resultados en 59 contratos; **ninguno accionable**. La tabla resume las categorías y su tratamiento.

CATEGORÍA	EJEMPLOS	TRATAMIENTO
Patrones por diseño	Comparaciones con <code>block.timestamp</code> , ensamblador, llamadas de bajo nivel, calls-en-bucle (batch)	Aceptado
Falsos positivos	Mapping “no inicializado”; reentrancy en <code>createWallet</code> (solo un <code>emit</code> tras un <code>setGuardians</code> de confianza)	Sin acción
Código muerto — <code>_feeNumerator()</code>	Marcado como no usado	Falso positivo — verificado en uso (ver nota)
Falta zero-check — <code>owner_</code> de la <code>factory</code>	El constructor asigna <code>owner</code> sin validar	Corregido
Cosmético	Nomenclatura, dígitos literales, complejidad ciclomática	Aceptado

Nota de verificación — un hallazgo de herramienta no es un hecho. Slither marcó `_feeNumerator()` como código muerto eliminable. Una comprobación del código mostró que **sí** se usa para calcular la comisión (base 0,05% / override del agente 0,15%); eliminarlo habría roto la lógica de comisiones. El hallazgo era un falso positivo causado por el dispatch de funciones virtuales y **no** se actuó sobre él.

Remediación aplicada

Se añadió un control defensivo de dirección cero — `if (owner_ == address(0)) revert ZeroOwner();` — al constructor de ambas `factories`, sincronizado en los dos repositorios. Suite completa en verde: **forge test = 128/128** en cada repo. Slither se re-ejecutó sobre los contratos V2 tras el endurecimiento de gas (§9): **0 resultados**; la suite unitaria queda en **131/131**.

4 Hallazgo — Bypass de caps de token vía `approve`

ALTA

CORREGIDO Y REDESPLEGADO

Descripción

Los presupuestos **diario** y **total** por token del agente podían saltarse por completo. La validación contabilizaba el gasto de `transfer`, pero cargaba `approve` solo contra el cap por transacción — no contra el diario ni el total, y sin restringir el spender. El agente podía por tanto `approve` un allowance (solo capeado por per-tx) y vaciarlo con un `transferFrom` externo, sin tocar nunca los contadores acumulados.

Prueba de concepto (ejecutada on-chain, presupuestos agotados a diario 2/2 · total 2/3)

PASO	ACCIÓN	RESULTADO
1	<code>executeAsAgent → USDC.approve(B, 1)</code>	Pasa — per-tx OK, diario ignorado
2	Como EOA: <code>USDC.transferFrom(wallet, B, 1)</code> (fuera de <code>executeAsAgent</code>)	1 USDC extraído; contadores sin cambios

Un transfer plano del mismo importe sí revirtió correctamente con `TokenDailyLimitExceeded`; `approve` no. Al no tener límite diario, el ciclo `approve(1) → transferFrom(1)` era repetible — vaciando la wallet más allá de los caps diario (2) y total (3). Los fondos de la PoC se devolvieron a la wallet.

Remediación

En el bucle de acumulación de `executeAsAgent` se extendió la condición con `|| _isApprove(batch[i].callData)` de modo que `_applyTokenSpend` carga los `approve` contra los mismos caps diario/total y actualiza los contadores. La elección es deliberadamente conservadora (un `approve` sobrescribe el `allowance`, por lo que puede sobre-contar — la dirección segura). Se añadieron seis tests de regresión; **5 fallan sin el fix, todos pasan con él.**

Superficie acotada. Los tokens aceptan solo `transfer` y `approve` por selector; `transferFrom` e `increaseAllowance` están bloqueados. ETH no tiene primitiva de delegación, así que no se ve afectado. `approve` era el único vector — ahora cerrado y re-verificado tras el redesplicue (ver §6).

5 Despliegue de la remediación

Ambas factories cambiaron de bytecode (zero-check), produciendo nuevas direcciones deterministas CREATE2 en Arbitrum Sepolia. La configuración de red del frontend se actualizó en consecuencia.

CONTRATO	NUEVA DIRECCIÓN (PARCHEADA)	ANTERIOR (DEPRECADA)
AgentWalletFactory	0xc87aa10747A92B472EF6B36e190B84c897a2953e	0xe7ad...7e04
SmartWalletFactory	0xa5290A51a73903176e09C864E1542a07da67BD12	0x6890...F58d

Separación de privilegios

- **Deployer (sin control):** 0xA3e06B6443D911915bb4eD157832d43C25D6617D — el owner de la factory se fija desde un argumento del constructor, no desde `msg.sender`, así que el deployer no tiene privilegios.
- **Admin / kill-switch:** 0x3145Bd5e2489d8bDdAb17F23F26F07Ac5aD55A4c — inmutable; la única acción privilegiada es el `kill()` de un solo sentido.

Tamaños runtime dentro del límite EIP-170 (AgentWallet 21.169 B; AgentWalletFactory 23.803 B — 773 B de margen).

6 Batería de pruebas de permisos del agente

Ejecutada contra la wallet parcheada 0x1ED261...F0C3 con el EOA del agente 0x3852...4aB4. Reverts confirmados por simulación; casos válidos como transacciones reales. Los destinatarios permitidos fueron los dos guardianes de recovery (controlados por el bot, fondos recuperables).

#	PRUEBA	ESPERADO	RESULTADO
1	ETH per-tx (0,0002 > máx 0,0001)	revierte	ExceedsPerTxLimit
2	ETH dentro de límite (0,0001)	pasa	Success
3	USDC per-tx (2 > máx 1)	revierte	ExceedsTokenMaxAmount
4	USDC diario (1+1+1, cap 2) — txs reales	la 3ª revierte	TokenDailyLimitExceeded
5	Envío de token a destino no permitido	revierte	RecipientNotAllowed
6	Envío de ETH a destino no permitido	revierte	RecipientNotAllowed
7	Presupuesto total (0,0002+0,0001 = 0,0003) — txs reales	la siguiente revierte	AgentBudgetExceeded
8	Presupuesto período (0,0003+0,0002 = 0,0005) — txs reales	la siguiente revierte	PeriodBudgetExceeded
9	Auto-rollover del período (periodStart 0)	la 1ª tx resetea la ventana	Verificado
10	Pausado (el owner pausa)	agente bloqueado	EnforcedPause
11	Expiración (caducó sola)	agente bloqueado	AgentPermissionsExpired

Observación. La lista blanca de destinatarios aplica también a los **transfers de token**, no solo a ETH — un envío de token a una dirección no permitida revierte `RecipientNotAllowed`. La re-

autorización vía "Edit" no resetea el contador diario (indexado por día UTC); con `periodStart = 0` la primera operación rota la ventana del período.

Resistencia a escalada de privilegios (de la sesión del 06-06)

Un agente que apunta a la propia wallet para llamar a funciones de owner (`pauseAgents`, `revokeAgent`, `authorizeAgent`) revierte con `AgentCannotCallWallet`; las llamadas arbitrarias a contratos fuera de la lista de protocolos están bloqueadas. El agente solo puede mover ETH y tokens permitidos hacia destinatarios permitidos.

7 Correcciones de soporte (frontend)

Se corrigieron tres incidencias en la aplicación web de la wallet, sincronizadas con el repositorio público.

ÁREA	INCIDENCIA	CORRECCIÓN
lib/wallet.ts	La recuperación de credencial consultaba solo el evento de la factory base, por lo que reabrir una wallet <i>de agente</i> por dirección fallaba (“no hay wallet activa”).	Consultar también agentFactory / AgentWalletCreated.
Página de agentes	Los límites por token se mostraban solo como texto.	Leer getTokenSpent on-chain; pintar barras de progreso Diario/Total (verificado en vivo).
Flujo de deploy	El gas que pasaba la app infraestimaba el deploy de la agent wallet (~21 KB) en Arbitrum, provocando fallos al firmar.	Delegar en la estimación de red de la wallet.

8 Recuperación por Guardianes — Ejecutada y Verificada

La wallet base admite recuperación social: un quórum de 2 de 3 guardianes puede rotar la passkey del propietario tras un timelock. Se probó de extremo a extremo en la wallet 0xEBc851bA9e8Ce32A2cb8eB99544F5F024f06E61d y ya está **completada**.

Proceso

- **Inicio (2026-06-06).** Recovery propuesta hacia una nueva passkey de propietario; los guardianes 1 y 2 (0xa337...5908, 0x8e06...2Ccc) aprobaron — quórum 2/2.
- **Timelock.** recoveryReadyAt on-chain = 2026-06-08 17:24 UTC; durante toda la ventana el propietario conservó la capacidad de cancelar.
- **Ejecución (2026-06-08).** Transcurrido el timelock se llamó executeRecovery() desde la UI /recover. El signer on-chain rotó a la nueva passkey (X 0x303591... / Y 0xb858ed...), reemplazando al propietario anterior (X 0x070f7a...).
- **Comprobación positiva.** El nuevo propietario firmó de inmediato una transferencia real de **19 USDC** con el nuevo Face ID — confirmada on-chain.

Prueba negativa — la passkey antigua queda anulada

Para confirmar que la rotación revocó realmente la clave anterior, se intentó una transferencia con la passkey **antigua**, cuya credencial WebAuthn sigue presente en el autenticador de pruebas. La firma se genera localmente, pero debe validarse on-chain contra el propietario actual.

#	ACCIÓN	ESPERADO	RESULTADO
1	Enviar 0.0001 ETH, UserOp firmado por la passkey ANTIGUA	rechazo en validación	FailedOp(0, AA24 signature error)
2	Efecto on-chain (saldos ETH / USDC y nonce)	sin cambios	Sin cambios

Resultado. La recovery se completó con éxito: la propiedad pasó a la nueva passkey, la nueva clave opera la wallet y la antigua es rechazada criptográficamente por el EntryPoint durante la validación de firma — ya no puede mover fondos aunque su credencial local siga existiendo.

9 V2 — Endurecimiento de gas en mainnet y despliegue multichain

ALTA (disponibilidad / sobrepago)

CORREGIDO — V2 DESPLEGADA 2026-06-11

Descripción

En Arbitrum mainnet, los swaps a través de la wallet fallaban en la estimación de gas. La lógica de comisiones del Caso 3 escanea el calldata buscando candidatos a token y sondea cada uno con `balanceOf`. Los precompiles del sistema de Arbitrum (`0x64-0x6f`) reportan bytecode (`0xfe`), por lo que el control por longitud de código no los filtraba — y llamarlos consume **todo el gas reenviado** en lugar de revertir barato. El estimador subía el límite de gas hasta ~32M para que la sonda “cupiera”; ese límite inflado es lo que la wallet habría prefinanciado (riesgo de sobrepago 64×) y, en la práctica, la operación fallaba.

	ANTES (BUG)	DESPUÉS (FIX V2)
Estimación de gas (frontend / bundler)	~32.000.000	~400.000
Efecto de un candidato quema-gas	quema todo el gas reenviado	capado a 100k, la ejecución continúa
Resultado	el swap falla (o sobrepago 64×)	swap normal, ~\$0,02–0,05 en Arbitrum

Remediación (V2)

- **Sondas capadas en gas.** Una constante compartida `PROBE_GAS_CAP = 100_000` capta cada `staticcall` de `balanceOf` tanto en `_tryBalanceOf` (snapshot) como en `_collectFeesOnIncrease` (cobro de comisión). El peor caso de un escaneo completo queda acotado a ~2,2M de gas frente a los 32M anteriores; un ERC-20 real necesita mucho menos que el cap, así que la contabilidad de comisiones no se ve afectada.
- **El cobro de comisión nunca puede romper una llamada del usuario.** Si un token se comporta mal tras el swap, su comisión se omite en lugar de revertir la transacción del usuario.
- **Cota contable explícita.** `executeAsAgent` ahora exige `totalEth ≤ type(uint128).max` antes del cast, eliminando un caso límite de truncamiento silencioso para agentes configurados sin límites de ETH.

Verificación

- **forge test = 131/131** — tests de regresión añadidos: un candidato quema-gas no deja sin gas al swap; un `balanceOf` que revierte se filtra sin ser fatal; un token con retorno sobredimensionado se snapshotea igualmente.
- **Slither 0.11.3 re-ejecutado sobre V2: 0 resultados.** (Nota de herramienta: Slither no puede generar IR para los internos del `ReentrancyGuard` con `transient storage` de OpenZeppelin — limitación conocida del parser, no un hallazgo.)
- **Tamaños EIP-170:** AgentWallet 21.214 B; AgentWalletFactory 23.848 B (728 B de margen); SmartWallet 13.432 B; SmartWalletFactory 16.030 B.

Despliegue V2 — CREATE2 determinista, misma dirección en todas las redes

CONTRATO	DIRECCIÓN (IDÉNTICA EN TODAS LAS REDES)
----------	---

SmartWalletFactory V2	0x230b7010529AB6977Dd8581B3eF018ef865BdEf1
AgentWalletFactory V2	0x8D9e24022777173AD6336e00884b6C87c7EF054c

Desplegadas el 2026-06-11 en Arbitrum One (42161), BNB Chain (56) y Arbitrum Sepolia (421614). Las wallets creadas por factories pre-V2 no reciben el fix — las wallets nuevas deben crearse a través de las factories V2.

10 Conclusión

La BVCC Agent Wallet aplica un modelo de permisos completo y por capas. Se identificó un bypass de severidad alta, se reprodujo on-chain, se remedió, se redesplegó y se re-verificó. El análisis estático no reveló más incidencias accionables. La batería completa de pruebas on-chain pasa sobre el bytecode parcheado y no queda ninguna incidencia abierta. Adicionalmente, la recuperación por guardianes se ejecutó y verificó de extremo a extremo — rotación de propiedad a una nueva passkey y rechazo on-chain de la clave antigua (ver §8). Una ronda V2 posterior corrigió un problema de gas en mainnet en el snapshot de comisiones, se verificó con tests y una re-ejecución limpia de Slither, y concluyó con el despliegue determinista de las factories V2 en Arbitrum One, BNB Chain y Arbitrum Sepolia (ver §9).

Anexo — identificadores clave

ELEMENTO	VALOR
Red	Arbitrum Sepolia (421614)
Wallet parcheada	0x1ED261BA084c4E6117daf289f3ca233BBA3cF0C3
Agente (rol B)	0x38529C66F3cf22453D66B9E2A20FdF2676544aB4
AgentWalletFactory (Sepolia, pre-V2)	0xc87aa10747A92B472EF6B36e190B84c897a2953e
SmartWalletFactory (Sepolia, pre-V2)	0xa5290A51a73903176e09C864E1542a07da67BD12
SmartWalletFactory V2 (todas las redes)	0x230b7010529AB6977Dd8581B3eF018ef865BdEf1
AgentWalletFactory V2 (todas las redes)	0x8D9e24022777173AD6336e00884b6C87c7EF054c
Redes V2	Arbitrum One (42161) · BNB Chain (56) · Arbitrum Sepolia (421614)
Herramientas	Slither 0.11.3 · Foundry (forge 1.5.1) · viem 2.x

Confidencial — Block Venture Chain Capital. Preparado para revisión interna e inclusión en el repositorio. Las pruebas de seguridad se realizaron en una testnet pública (Arbitrum Sepolia); las factories V2 están desplegadas además en Arbitrum One y BNB Chain.